

Practical Work No. 1 (PW1)

Data Models in Database Systems, 2025

General information:

Group number:	DMDS PW group 15
Variant (domain):	ARENA
Link to the video:	https://drive.google.com/file/d/1ifmngp3nCwkBHsZKrdAcwdXSy5U2cXmy/view?usp=drive_link
Link to the LiveSQL script:	https://livesql.oracle.com/ords/livesql/s/d4246cwlbd1cxa96d0nujqmos

Group information:

#	Role	Full name	Student ID number
1	Administrator	Ngoc Bao Tram Tran	231ADB294
2	Member	Ilaha Habibova	231ADB231

PROJECT PASSPORT

TABLES

Criterion	Name of the table	LiveSQL Statement No.			Score
		CREATE	INSERT	SELECT	
Table with row type objects	PEOPLE	4	5	6,7,44	
	SEATS	10	11	12	
	EVENTS	15	18	19,25,46	
	ORDERS	30	33,34,47	35,48,51	
	REQUESTS	23	26	27	
	SEASON_PASSES	38	40,41	42	
Table with object columns	PEOPLE	4	5	6,7,44	
Table with object collections	ORDERS	30	33,34,47	35,48	
Table of other type	-	-	-	-	

FUNCTIONS & OPERATORS

Function	LiveSQL Statement No.	Score
VALUE()	19,35,48	
TABLE()	31,32,35,48, 51	
REF()	18,24,33,34,35,42,47,48	
DEREF()	16,21,25,27,32,42	
TREAT()	19,21	
IS OF type	19,35,48	

OBJECT METHODS

Criterion	Type name and function or procedure name	LiveSQL Statement No.			Score
		Declare	Define	Test	
Member function	PERSON get_age	3	43	44	
	EVENT get_description	8	45	46	
	SPORT_EVENT get_price_for_seat	13	16	19	
	SPORT_EVENT get_description	13	16	46	
	OTHERS get_price_by_sector	14	17	19	
	OTHERS get_description	14	17	46	
	TICKET get_price_for_ticket	20	21	35,36,48	
Member procedure	CUSTOMER subscribe	1	39	40,41	
	EVENT_ORGANIZER submit_request	2	24	26	
MAP member function	TICKET_ORDER get_total_price	29	32	49	
ORDER member function					
Static function					

Criterion	Type name and function or procedure name	LiveSQL Statement No.			Score
		Declare	Define	Test	
Static procedure	TICKET_ORDER add_order	29	32	50,51	
Function returning SYS_REFCURSOR	TICKET_ORDER view_tickets	29	32	36	

OTHER CONSTRUCTIONS

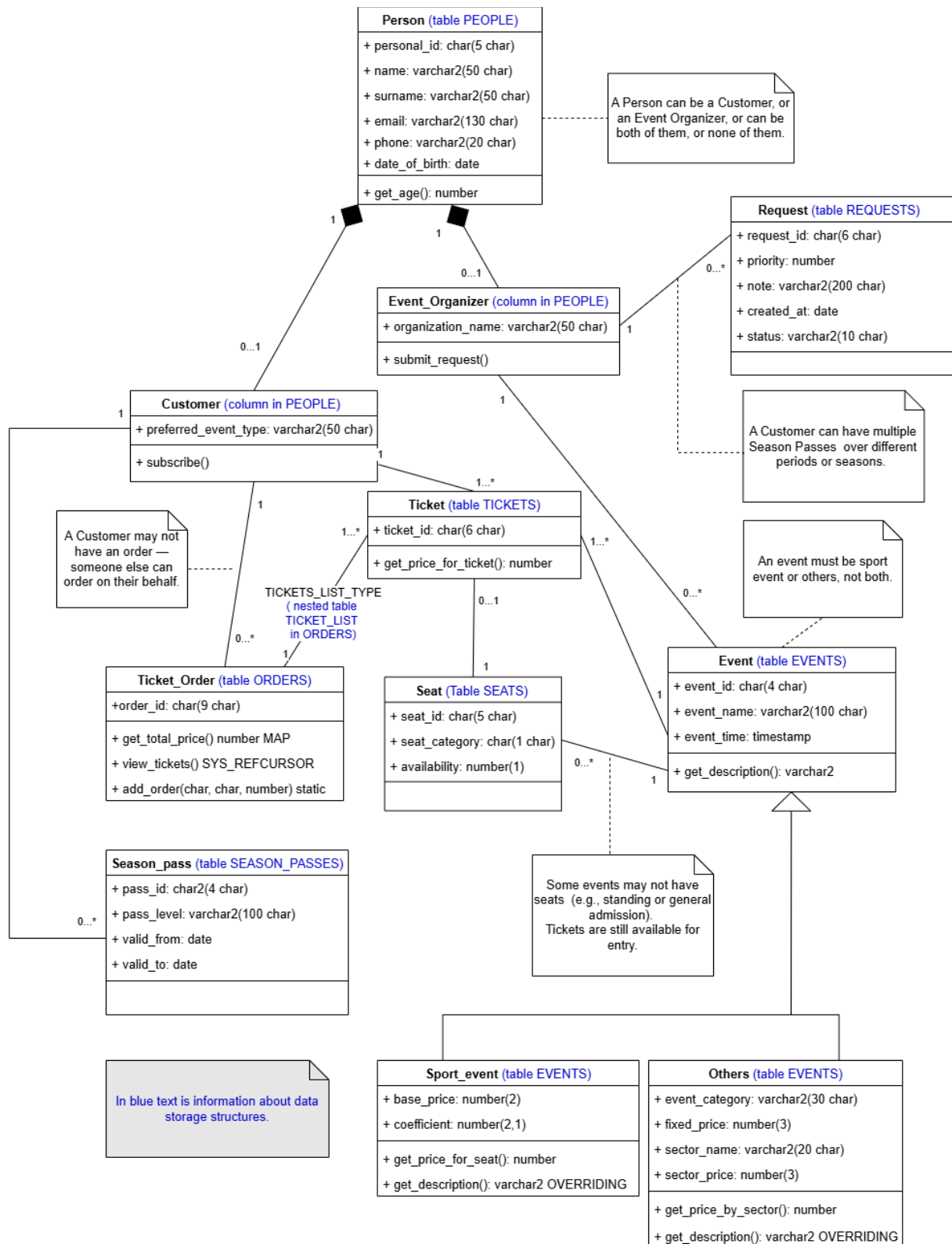
Type inheritance

- SPORT_EVENT UNDER EVENT – statement No. 13
- OTHERS UNDER EVENT – statement No. 14

Function or procedure overriding

- SPORT_EVENT – get_description – statement No. 13
- OTHERS – get_description – statement No. 14

CLASS DIAGRAM



1. Person:

This class represents general information about individuals interacting with the arena system. It stores shared personal attributes such as ID, name, contact information, and birth date, and includes the method - member function `get_age()`.

Each person can independently play one or both roles - or neither - *Customer* and/or *Event_Organizer*.

Customer and/or Event Organizer — represented as object columns within the *PEOPLE* table rather than as inheritance subtypes of *Person*.

2. **Customer and Event Organizer:**

These roles describe specific behaviors and attributes of *Person*.

a. Customer includes attributes such as `preferred_event_type` and the member procedure `subscribe()`.

b. Event Organizer includes organizer-specific data like `organization_name` and the member procedure `submit_request()` for submitting event requests.

These role columns allow REF relationships from other tables (such as *ORDERS* or *EVENTS*) to a specific person in a defined role.

3. **Event:**

This class stores general event details (`event_id`, `event_name`, `event_time`) and provides the main method — member function `get_description()`.

One event can have zero or many seats, because some events are standing-only and do not have assigned seats.

One event can have one or many tickets.

One event belongs to only one event organizer.

Other than general events, events are specialized into Sport Events and Other Events through inheritance:

a. **Sport Event** adds pricing parameters (`base_price`, `coefficient`) and a method — member function `get_price_for_seat()`, and it overrides `get_description()` of *Event* to specify sport event details.

b. **Others**, such as concerts, have attributes like `event_category`, `sector_name`, and `sector_price`, and include the member function `get_price_by_sector()` and an overridden `get_description()`.

4. **Seat:**

Demonstrates the arena's seats, with attributes such as `seat_id`, `seat_category`, and `availability`.

One seat can belong to one ticket for one specific event.

5. **Ticket:**

Identified by `ticket_id`, it indicates a personalized ticket linked to one seat and one event.

The member function `get_price_for_ticket()` calculates the final ticket price based on seat properties and event pricing logic (e.g., $\text{sport} = \text{base} \times \text{coefficient} \times \text{category}$; others = sector-based pricing).

Each event can have many tickets, while each ticket refers to exactly one seat and one event.

6. **Ticket_Order:**

Ticket_Order can contain one or more tickets and is identified by `order_id`.

It includes a nested table *TICKETS_LIST_TYPE* (stored as *TICKETS* in *ORDERS*) to store multiple tickets under the same order.

It has methods like map member function `get_total_price()` to compare orders by total cost, member function `view_tickets()` returning *SYS_REFCURSOR* for ticket lists, and static procedure

`add_order(p_customer_id, p_event_id, p_ticket_qty)` to create new orders.

Each *Ticket_Order* is linked to exactly one customer via a REF column, but a customer may have zero or more *Ticket_Orders*.

7. **Season Pass:**

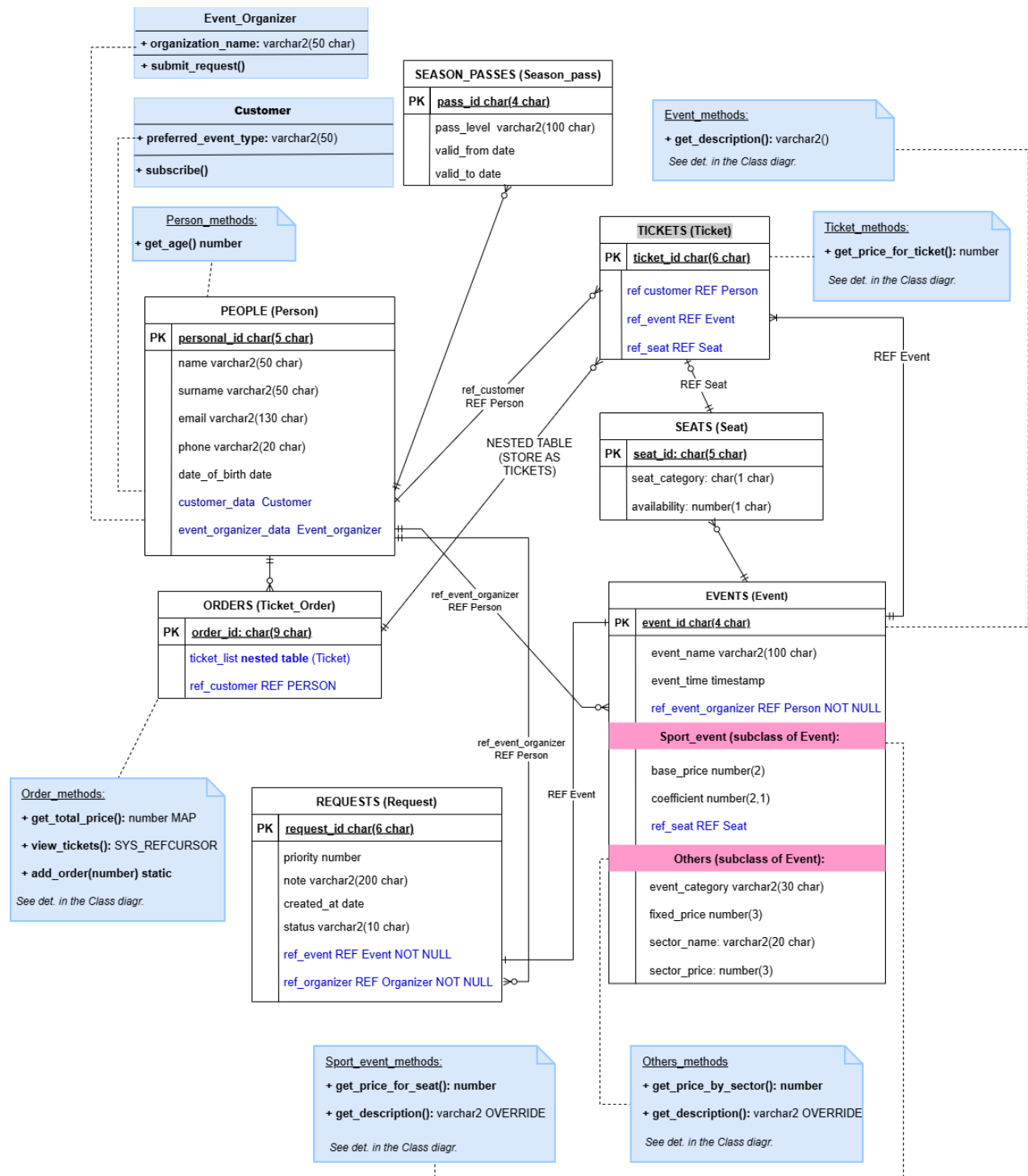
It stores season pass details with attributes such as pass_id, pass_level (Silver, Gold, VIP), and validity period.

Pass levels define event access (e.g., Silver → Silver events, Gold → Silver & Gold, VIP → all).

8. **Request:**

Captures organizer requests related to events. Each request references one event and one organizer (both via REF columns) and includes priority, note, created_at, and status. This supports a review workflow where admins approve or reject requests.

EXTENDED ER DIAGRAM



GROUP WORK REPORT

• How was the practical work developed?

First, we reviewed the professor's sample, our given description (ARENA), the assignment description, and the grading criteria, and tried to understand the parts that were unclear to us by going through the lecture materials again. Then, we began developing the class diagrams. While creating the diagrams, we had to think about which parts of the given description (ARENA) we could apply directly and which parts needed modification.

After that, when we developed the ER diagram, we noticed some changes were needed in the class diagram, so we re-edited it. After that, we checked whether we had fulfilled all the requirements; if not, we made additional adjustments. Finally, we started coding.

During the coding process, we also observed which parts from the diagrams could be easily applied and which ones required rethinking. For example, at the beginning, we defined the nested table **Tickets** in both the class and ER diagrams as a table with object collections, but when implementing it in the code, I reviewed the theory and realized it could be applied differently. So, I modified the class and ER diagrams to make them more applicable, as shown in the final results. We then fixed the remaining errors and completed the code.

• How did you split the work?

Ilaha and Tram met at the library and created the Class and ER diagrams together (at the same time, each working on different objects). For the coding part, we decided to work from home and divide the tasks into two parts: Events, Seats, Tickets were handled by Ilaha, while People, Season_Passes, Requests, and Orders were handled by Tram.

We both needed the People table initially to start creating other tables due to dependencies. At the beginning, Tram started with the Person type and PEOPLE table, after which Ilaha worked on her assigned parts. Tram then used Ilaha's code as a baseline. Later, we met again to review everything, make adjustments, and finalize the structure. Tram formatted the code and combined all parts into one complete script.

Meanwhile, Ilaha worked on the submission document, which Tram later reviewed and modified. Throughout the process, we discussed every detail together using online tools and shared files, since each person's work affected the other's, and everything needed to function as one integrated system.

• What was productive, and what was not?

Collaboration was very successful and led to productive work because we learned from each other. However, we found LiveSQL not very user-friendly and had to spend much more time working with it.

• What problems, if any, arose and why?

Actually, there was a problem with the code — when we split it like that, it was hard to know which part (Event, Ticket, etc.) to implement at which statement order because they were dependent on each other. But then, Ilaha provided the baseline (although at that time, the code was not yet complete or entirely correct). Still, it helped a lot with the process. When she gave the code to Tram, Tram didn't understand it at first and had to figure it out. At that time, Ilaha shared her code in Notion because LiveSQL is a tool that's difficult to use and doesn't allow two people to work simultaneously. So, Tram had to modify Ilaha's code to make it fit with her own, and after that, the process flow became smoother.

CONCLUSIONS

Within this project we modelled the core entities, relationships, and behaviors required for managing events, customers, organizers, and ticketing operations within an arena. We used object-oriented features like inheritance, methods, and references. The system has different event types with specialized

pricing through subtype hierarchies. Object tables and nested tables handled complex relationships between people, events, and tickets.

In the system, we implemented constraints, triggers, proper reference handling and VALUE(), TREAT(), TABLE() functions.

Overall, the implementation showed how object-relational modeling can represent real-world scenarios.

SOURCES AND TOOLS

Sources:

1. UML Class Diagram - geeksforgeeks.org – link: <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/>
2. ER Diagram Symbols and Notation - lucidchart.com – link: <https://www.lucidchart.com/blog/er-diagram-symbols-and-notation>

Tools:

1. For Diagrams: draw.io
- Link of our work: <https://drive.google.com/file/d/10c3jSydNtKRSsVwNwxyU6zmsomBf9vFB/view?usp=sharing>
2. For coding: LiveSQL